

Experiment #07

Implementation of Vector Operations using OpenMP Sections

Objective:

Objective of this lab is to get the knowledge about OpenMP parallel computation techniques.

Software Tools:

- Ubuntu

Theory:

OpenMP "sections"

The sections construct is a noniterative work-sharing construct that contains a set of structured blocks that are to be distributed among and executed by the threads in a team. Each structured block is executed once by one of the threads in the team in the context of its implicit task.

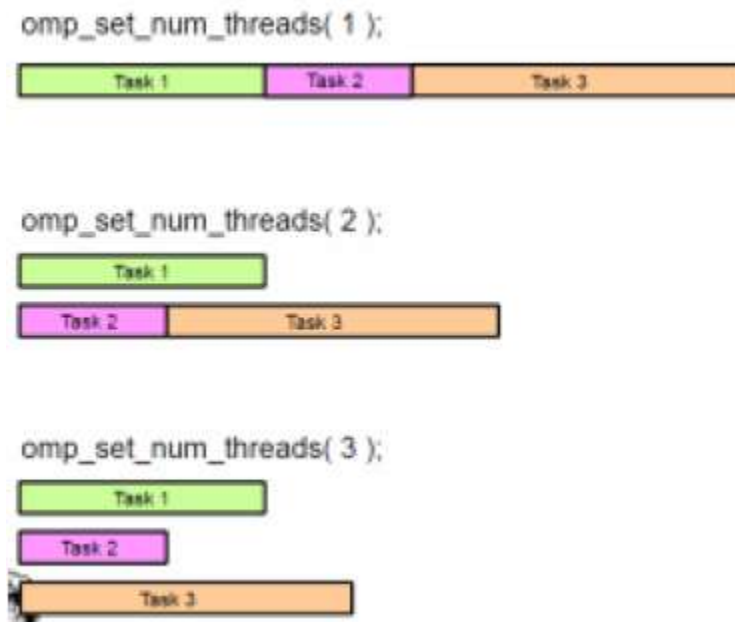


Figure: 1

The OpenMP sections directive is a quick way to thread different functional blocks. The section of code in below figure shows how easy it is to break up execution of four different functions into two threads. Here it takes only three additional lines of code (not counting enclosing braces) to thread these functions.

While this is a convenient way to do functional threading, there is a drawback: the number of threads OpenMP creates is limited to the number of section groups defined or processors available, whichever is less. So even if the code in the example was run on a system with four processors, only two processors would be utilized. Conversely, if there are more section blocks than processors, OpenMP will decide how to schedule the blocks for execution, which may not be optimal.

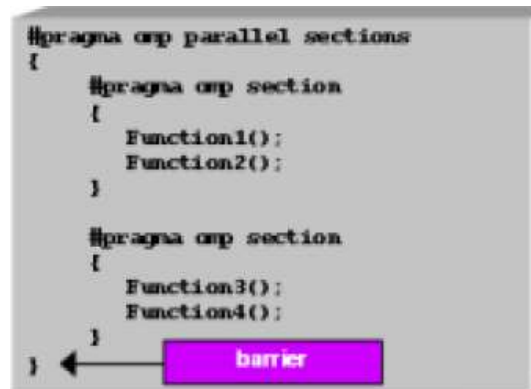


Figure: 2

Barrier in the above figure shows where the application waits for all the threads to complete execution.

OpenMP sections Directive: Example

sec_exp1.cpp:

```

#include <omp.h>
#include "iostream"
#define N 10
using namespace std;
int main ()
{

#pragma omp parallel sections
{
    #pragma omp section
    {
        cout<<"\nSection 1 id = "<< omp_get_thread_num();
    }

    #pragma omp section
    {
        cout<<"\nSection 2 id = "<< omp_get_thread_num();
    }

    #pragma omp section
    {

```

```

        cout<<"\nSection 3 id = "<< omp_get_thread_num();
    }
}
}

```

In this example, we simply created three parallel sections which executed on three different threads. Each section prints the thread number which runs the respective section code.

sec_exp2.cpp:

```

#include <omp.h>
#include "iostream"
using namespace std;
#define N 100

int main ()
{
    int i, nthreads, tid;
    float a[N], b[N], c[N], d[N];

    /* Some initializations */
    for (i=0; i<N; i++) {
        a[i] = i * 1.5;
        b[i] = i + 22.35;
        c[i] = d[i] = 0.0;
    }

    #pragma omp parallel sections
    {
        #pragma omp section
        {
            cout<<"Thread " << omp_get_thread_num() << " doing section 1\n";
            for (i=0; i<N; i++)
            {
                c[i] = a[i] + b[i];
                cout<<"Thread "<<omp_get_thread_num()<< " : "<< i <<" = "<< c[i] <<endl;
            }
        }

        #pragma omp section
        {
            cout<<"Thread "<< omp_get_thread_num() << " doing section 2\n";
            for (i=0; i<N; i++)
            {
                d[i] = a[i] * b[i];
                cout<<"Thread "<< omp_get_thread_num() << " : "<< i <<" = "<< d[i] <<endl;
            }
        }
    }
}

```

```

    }

    } /* end of sections */

} /* end of parallel section */

}

```

In this example we do addition and multiplication of two arrays using two omp sections which basically runs on two separate threads.

sec_func.cpp:

```

#include "iostream"
#include <math.h>
#include <omp.h>

using namespace std;

int frequency_of_primes (int size) {
    int i,j;
    int n = size;
    int freq=n-1;
    for (i=2; i<=n; ++i) for (j=sqrt(i);j>1;--j) if (i%j==0) {--freq; break;}
    return freq;
}

int main()
{
    int i, g;
    int size = 100000;

#pragma omp parallel sections
    {

#pragma omp section
    {
        cout<<"Thread "<< omp_get_thread_num() << " doing section 1\n";
        g = frequency_of_primes (size);
        cout<<"\nThread "<< omp_get_thread_num() << ":prints freq of prime number: "<< g <<endl;
    }

#pragma omp section
    {
        cout<<"Thread "<< omp_get_thread_num() << " doing section 2\n";
    }
    }
}

```

```
}
```

In this example we called a function in a section block which calculates the frequency of prime numbers.

Example.cpp

```
#include <omp.h>
#include "iostream"
#include "Timer.h"

using namespace std;

#define mx 5000

int vec1[mx];
int vec2[mx];
int vec3[mx];

void generateVectors()
{
    int i,j;
    for(i=0;i<mx;i++)
    {
        vec1[i]=rand()%100;
        vec2[i]=rand()%100;
    }
    /*
    for(i=0;i<mx;i++)
    {
        cout<<vec1[i]<<" ";
    }
    cout<<endl<<endl;
    for(i=0;i<mx;i++)
    {
        cout<<vec2[i]<<" ";
    }
    */
}

void addVectors()
{
    int i;
    for(i=0;i<mx;i++)
    {
        vec3[i]=vec1[i]+vec2[i];
    }
    /*
    for(i=0;i<mx;i++)
    {
        cout<<vec3[i]<<" ";
    }
    */
}
```

```

    }
    */
}

void subVectors()
{
    int i;
    for(i=0;i<mx;i++)
    {
        vec3[i]=vec1[i]-vec2[i];
    }
    /*
    for(i=0;i<mx;i++)
    {
        cout<<vec3[i]<<" ";
    }
    */
}

int main ()
{

    #pragma omp parallel sections
    {
        #pragma omp section
        {
            // cout<<"\nSection 1 id = "<< omp_get_thread_num();
            addVectors();

        }

        #pragma omp section
        {
            // cout<<"\nSection 2 id = "<< omp_get_thread_num();
            subVectors();
        }
    }

    return 0;
}

```

In this example we called two functions in two different sections block which calculates the addition & subtraction of vectors.

Code: -

```
#include<iostream>
#include"Timer.h"
#include<omp.h>
const size_t N = 1e7;
double *a, *b, *sum, *diff;
using namespace std;
void init()
{
    for(size_t i=0; i<N; i++)
    {
        a[i] = i*2.1;
        b[i] = i + 3.24;
    }
}
void add()
{
    for(size_t i=0; i<N; i++)
    {
        sum[i] = a[i] + b[i];
    }
}
void sub()
{
    for(size_t i=0; i<N; i++)
    {
        diff[i] = a[i] - b[i];
    }
}
int main()
{
    Timer seqTime("Sequential Execution: ");
    // array allocation
    a = new double[N];
    b = new double[N];
    sum = new double[N];
    diff = new double[N];
    // serial execution
    init();
    seqTime.Start();
    add();
    sub();
    seqTime.Stop();
    seqTime.Print();
    //parallel execution
    /*
    init();
    thread t1(add);
    thread t2(sub);
    t1.join();
    t2.join();
    */
    Timer parTime("Parallel Execution: ");
    init();
```

```

parTime.Start();
#pragma omp parallel sections
{
    #pragma omp section
    {
        // cout<<"\nSection 1 id = "<< omp_get_thread_num();
        add();
    }

    #pragma omp section
    {
        // cout<<"\nSection 2 id = "<< omp_get_thread_num();
        sub();
    }
}
parTime.Stop();
parTime.Print();
return 0;
}

```

Output: -

```

fasih@fasih-Latitude-7280:~/pdclabs/lab07$ subl code.cpp
fasih@fasih-Latitude-7280:~/pdclabs/lab07$ g++ -o code.out -fopenmp code.cpp
fasih@fasih-Latitude-7280:~/pdclabs/lab07$ ./code.out
Sequential Execution: : 100.937 msec
Parallel Execution: : 49.375 msec
fasih@fasih-Latitude-7280:~/pdclabs/lab07$ 

```


Lab Task:

Write a code for the following using OpenMP

- Write a Program which converts the sec_exp2.cpp example's sections code into two different functions and call them in the parallel sections.
 - Write a program using sec_func.cpp example code and add another function that finds a factorial and call it in the separate omp parallel section block. Print the time of serial execution and parallel execution.
 - Write a program having four functions: addition, multiplication, division, and subtraction. Create four omp parallel sections blocks and, in each block, call each function in it.
 - Write a program using example.cpp code that implements addition & subtraction of two vectors. Print the time of serial and parallel execution.
 - Write a program, which we implement in Lab, Calculate speedup & draw graph between serial & parallel execution to visualize the speedup.
 - Write a C++ program to compute the dot product of two vectors. Print the time of serial execution and parallel execution. Use thread level parallelism using OpenMP. Calculate speedup and draw graph by implementing 2, 4, 6 and 8 threads.
-